Εργασία Νευρωνικά Δίκτυα

Όνομα: Andrea Tollumi
ΑΕΜ: 220170

Μέρος Α: Θεωρητικό Μέρος

1. Ποιος είναι ο ρόλος της συνάρτησης ενεργοποίησης σε ένα MLP; (Παραδείγματα συναρτήσεων, ιδιότητες και επιπτώσεις στην απόδοση του δικτύου.)

Η συνάρτηση ενεργοποίησης είναι βασικό κομμάτι σε ένα MLP (Multilayer Perceptron), γιατί δίνει στο δίκτυο τη δυνατότητα να μαθαίνει πιο πολύπλοκα και μη γραμμικά μοτίβα στα δεδομένα. Αν δεν υπήρχαν συναρτήσεις ενεργοποίησης, το δίκτυο θα έκανε μόνο γραμμικούς υπολογισμούς (σαν μια απλή ευθεία) και δεν θα μπορούσε να λύσει προβλήματα που δεν διαχωρίζονται γραμμικά, όπως για παράδειγμα το "moons dataset".

2. Ποια είναι η διαφορά μεταξύ overfitting και underfitting; (Πώς σχετίζονται με την επιλογή της αρχιτεκτονικής ενός MLP)

<u>Overfitting (Υπερεκπαίδευση)</u>

- Το μοντέλο μαθαίνει πάρα πολύ καλά τα δεδομένα εκπαίδευσης, ακόμα και τις θορυβώδεις λεπτομέρειες ή τα λάθη τους.
- Αποτέλεσμα: πολύ καλή απόδοση στα δεδομένα εκπαίδευσης, αλλά κακή γενίκευση σε νέα, αόρατα δεδομένα (δηλαδή, υψηλό σφάλμα στο validation/test set).
- Συνήθως συμβαίνει όταν το μοντέλο είναι πολύ μεγάλο/πολύπλοκο (π.χ. πολλά layers, πολλοί νευρώνες) σε σχέση με το μέγεθος ή τη δυσκολία των δεδομένων.
-

<u>Underfitting (Υποεκπαίδευση)</u>

- Το μοντέλο δεν μαθαίνει επαρκώς από τα δεδομένα εκπαίδευσης.
- Αποτέλεσμα: κακή απόδοση τόσο στα δεδομένα εκπαίδευσης όσο και στα νέα δεδομένα.
- Συνήθως συμβαίνει όταν το μοντέλο είναι πολύ απλό/μικρό για το πρόβλημα που προσπαθεί να λύσει (π.χ. λίγα layers, λίγοι νευρώνες).

Η αρχιτεκτονική ενός MLP, δηλαδή πόσα κρυφά επίπεδα και πόσους νευρώνες έχει, παίζει μεγάλο ρόλο στο πώς μαθαίνει το μοντέλο. Αν το δίκτυο είναι πολύ μεγάλο και πολύπλοκο, μπορεί να μάθει πολύ καλά τα δεδομένα εκπαίδευσης, ακόμα και τα λάθη ή τις τυχαίες λεπτομέρειες, και τότε λέμε ότι κάνει

υπερεκπαίδευση (overfitting). Αυτό σημαίνει πως, όταν δει νέα δεδομένα, δεν τα προβλέπει σωστά. Αν όμως το δίκτυο είναι πολύ μικρό ή απλό, δεν μπορεί να μάθει καλά τις σχέσεις στα δεδομένα και κάνει **υποεκπαίδευση (underfitting)**, δηλαδή δεν δουλεύει καλά ούτε στα δεδομένα εκπαίδευσης ούτε στα καινούργια. Γι' αυτό πρέπει να επιλέγουμε σωστά το μέγεθος και την πολυπλοκότητα του MLP, ώστε να μαθαίνει καλά και να μπορεί να γενικεύει σωστά.

3. Ποια είναι τα πλεονεκτήματα και τα μειονεκτήματα του MLP σε σχέση με άλλα μοντέλα όπως τα RBF δίκτυα; (Αναφέρατε περιπτώσεις όπου το MLP δεν είναι η κατάλληλη επιλογή.)

Ένα MLP (Πολυεπίπεδο Πεπερασμένο Αντιληπτικό) θεωρείται συχνά καλύτερο από ένα RBF (Δίκτυο με Ραδιοβάσεις Συναρτήσεις) γιατί είναι πιο ευέλικτο και ισχυρό. Τα MLP έχουν πολλά επίπεδα και μη γραμμικές συναρτήσεις ενεργοποίησης, που τους επιτρέπουν να μαθαίνουν πολύπλοκα μοτίβα στα δεδομένα. Αυτό τα κάνει κατάλληλα για μεγάλα και σύνθετα προβλήματα, όπου μπορούν να συλλάβουν βαθύτερες σχέσεις.

Επιπλέον, τα MLP μπορούν να μεγαλώσουν εύκολα προσθέτοντας περισσότερα επίπεδα και νευρώνες, κάνοντάς τα προσαρμόσιμα σε πολλές διαφορετικές εργασίες. Χρησιμοποιούνται ευρέως σε πολλούς τομείς όπως η όραση υπολογιστών, η αναγνώριση ομιλίας και η επεξεργασία φυσικής γλώσσας, εκμεταλλευόμενα πολλές βελτιώσεις στους αλγορίθμους εκπαίδευσης.

Αντίθετα, τα RBF έχουν πιο τοπική απόκριση και μπορεί να εκπαιδεύονται πιο γρήγορα σε απλούστερα ή μικρότερα σύνολα δεδομένων. Ωστόσο, για σύνθετα προβλήματα και μεγάλα σύνολα δεδομένων, τα MLP συνήθως αποδίδουν καλύτερα και γενικεύουν πιο αποτελεσματικά.

Αν έχεις ένα πολύ μικρό σύνολο δεδομένων με λίγα μόνο δείγματα, το MLP συνήθως δεν είναι η καλύτερη επιλογή. Τα MLP έχουν πολλούς παραμέτρους (βάρη και προκαταλήψεις) και χρειάζονται αρκετά δεδομένα για να μάθουν σωστά. Όταν τα δεδομένα είναι λίγα, το MLP μπορεί να υπερεκπαιδευτεί — δηλαδή να "θυμάται" τα δεδομένα εκπαίδευσης χωρίς να μπορεί να γενικεύσει σε νέα, άγνωστα δεδομένα.

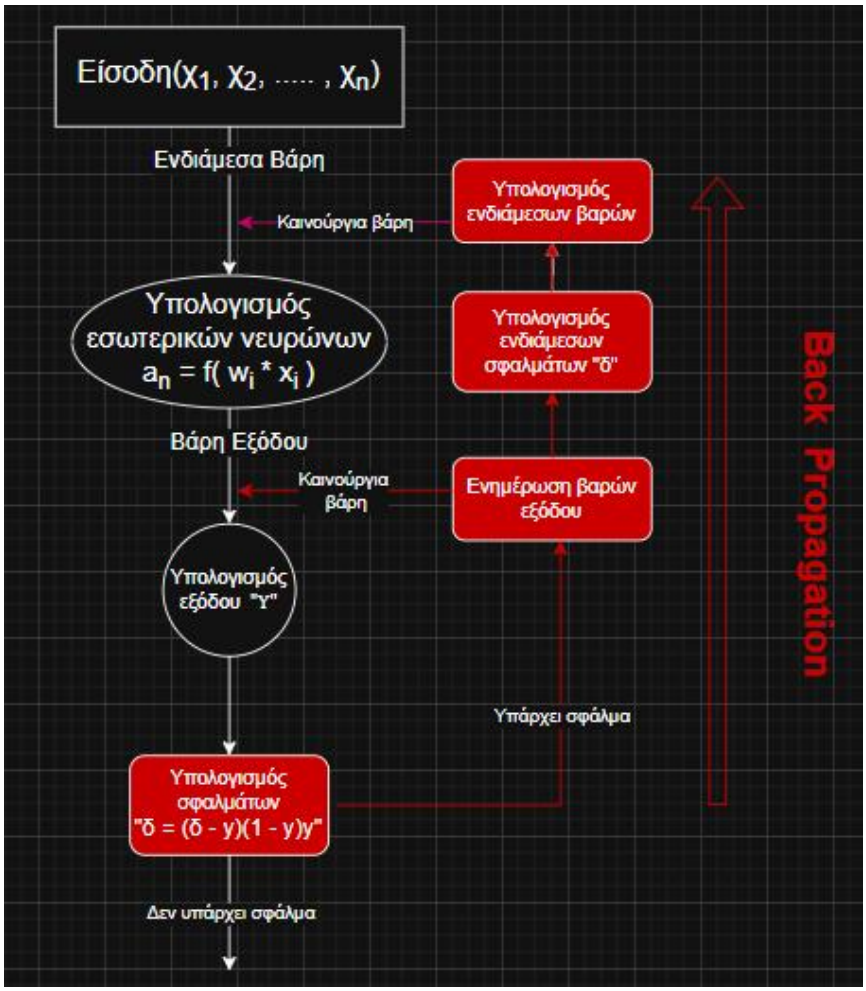
Σε τέτοιες περιπτώσεις, πιο απλά μοντέλα όπως η γραμμική παλινδρόμηση, τα δέντρα αποφάσεων ή ακόμα και τα RBF δίκτυα (που εστιάζουν σε τοπικά μοτίβα) μπορεί να αποδώσουν καλύτερα και να είναι πιο αξιόπιστα.

4. Ποια είναι τα βασικά στάδια του αλγορίθμου backpropagation; (Επεξηγήστε με χρήση διαγράμματος)

Η αντίστροφη διάδοση , είναι μια μέθοδος που χρησιμοποιείται για την εκπαίδευση νευρωνικών δικτύων . Στόχος της είναι να μειώσει τη διαφορά μεταξύ της προβλεπόμενης εξόδου του μοντέλου και της πραγματικής εξόδου προσαρμόζοντας τα βάρη και τις πολώσεις στο δίκτυο.
Λειτουργεί επαναληπτικά για να προσαρμόσει τα βάρη και την προκατάληψη για να ελαχιστοποιήσει τη συνάρτηση κόστους.

Τα βασικά στάδια της **backpropagation** (οπισθοδιάδοση σφάλματος) σε ένα τεχνητό νευρωνικό δίκτυο είναι τα εξής:

1. Προώθηση (Forward Pass)
2. Υπολογισμός Σφάλματος (Loss Calculation)
3. Οπισθοδιάδοση Σφάλματος (Backward Pass)
4. Ενημέρωση Βαρών (Weights Update)
5. Επανάληψη



Μέρος Γ: Πειραματική Αξιολόγηση & Αναφορά

Διαδικασία Υλοποίησης Βήμα προς Βήμα

Η υλοποίηση αφορά ένα πολυεπίπεδο νευρωνικό δίκτυο (MLP - Multilayer Perceptron).
Το μοντέλο σχεδιάστηκε για την επίλυση προβλήματος δυαδικής ταξινόμησης, χρησιμοποιώντας δεδομένα σε σχήμα μισοφέγγαρων (make moons dataset).
Για την υλοποίηση του νευρώνα χρησιμοποιούμε τα εξής εργαλεία/βιβλιοθήκες:

1. **import numpy as np:** Για υπολογιστικά μαθηματικά, πίνακες, αλγεβρικές πράξεις, στατιστική, κ.λπ.
2. **import matplotlib.pyplot as plt:** Για γραφήματα και οπτικοποίηση δεδομένων.
3. **from sklearn.datasets import make_moons:** Για να δημιουργήσουμε το dataset "moons".

➢ Προετοιμασία Δεδομένων

Αρχικά, δημιουργούμε ένα **δισδιάστατο dataset** χρησιμοποιώντας τη συνάρτηση **make_moons** από το πακέτο **sklearn.datasets.** Αυτή η συνάρτηση δημιουργεί 1000 σημεία στο χόρο σε σχήμα δύο μισοφέγγαρων, με τυχαία παραμόρφωση ώστε να μην σχηματίζουν τέλειες καμπύλες. Επίσης, ορίζει έναν σπόρο τυχαιότητας για αναπαραγωγιμότητα.

Πριν εκπαιδευτεί το μοντέλο, κάνουμε **κανονικοποίηση των δεδομένων** ώστε να έχουν την ίδια κλίμακα. Αυτό βοηθάει τον αλγόριθμο να εκπαιδεύεται πιο αποδοτικά και γρήγορα. Το πετυχαίνουμε υπολογίζοντας τον μέσο όρο κάθε χαρακτηριστικού του πίνακα X *(mean = np.mean(X, axis=0))*, υπολογίζοντας την τυπική απόκλιση κάθε χαρακτηριστικού του X *(std = np.std(X, axis=0))* και, τέλος, για κάθε τιμή στον πίνακα X, αφαιρούμε τον μέσο όρο και διαιρούμε με την τυπική απόκλιση.

Για την αξιολόγηση του μοντέλου, διαχωρίζουμε τα δεδομένα σε δείγματα που χρησιμοποιούμε για **εκπαίδευση (training)** και δείγματα για **δοκιμή (testing)**. Αυτό εξασφαλίζει ότι το μοντέλο θα ελεγχθεί σε δεδομένα που δεν έχει ξαναδεί. Θέτουμε έναν σπόρο με τιμή 42 (np.random.seed(42)) για να έχουμε πάντα την ίδια ακολουθία τυχαία ανακατεμένων αριθμών από το πλήθος των δειγμάτων στο dataset. Η επιλογή της τιμής του σπόρου δεν ήταν τυχαία, καθώς, σε σύγκριση με άλλους σπόρους που δοκιμάστηκαν (10, 20, 40 και 100), ο νευρώνας είχε καλύτερο **Accuracy**. Γίνεται τυχαία αναδιάταξη των δειγμάτων, ώστε να διασφαλιστεί η

τυχαία κατανομή των δεδομένων στις δύο κατηγορίες, και ορίζουμε το **80% των τυχαίων δειγμάτων για την εκπαίδευση** και το **20% για τη δοκιμή**.

➢ Κατασκευή του Νευρωνικού Δικτύου

Το επόμενο βήμα για την υλοποίηση του νευρώνα είναι η κατασκευή του (υλοποιείται μέσω της κλάσης MLP).
Η αρχιτεκτονική του νευρωνικού δικτύου που έχουμε δημιουργήσει έχει τα εξής χαρακτηριστικά:

1. **2 εισόδους**.
2. **2 κρυφές στρώσεις** από **10 νευρώνες η καθεμία**. *(Επειδή το πρόβλημα που έχουμε να λύσουμε δεν είναι γραμμικά διαχωρίσιμο, δύο κρυφές στρώσεις των 10 νευρώνων είναι αρκετές για την επίλυσή του. Επιπλέον, οι 10 νευρώνες με 100 εποχές δίνουν την καλύτερη ακρίβεια σε σύγκριση με άλλες δοκιμές που έγιναν.)*
3. **1 έξοδο**, διότι πρόκειται για δυαδική ταξινόμηση.

Για τον καθορισμό της εσωτερικής δομής του νευρωνικού δικτύου:

Δημιουργούμε μια λίστα, layer_size, που περιέχει τους αριθμούς νευρώνων για κάθε επίπεδο: το επίπεδο εισόδου, τα κρυφά επίπεδα και το επίπεδο εξόδου. Υπολογίζουμε τον αριθμό των επιπέδων σύνδεσης, **self.num_layers** (τα οποία είναι ένα λιγότερο από τον αριθμό επιπέδων).
Ορίζουμε τη **συνάρτηση ενεργοποίησης** και την παράγωγό της που θα χρησιμοποιηθεί στα κρυφά επίπεδα (**self.activation** και **self.activation_deriv**).

Για την εκπαίδευση του νευρώνα έχω χρησιμοποιήσει δύο συναρτήσεις ενεργοποίησης:

1. **ReLU** για τους κρυφούς νευρώνες. Το Moon Dataset απαιτεί περίπλοκα μη-γραμμικά όρια για να διαχωριστεί, και η ReLU βοηθάει το δίκτυο να μοντελοποιήσει αυτές τις σχέσεις.
2. **Sigmoid** για τον εξωτερικό νευρώνα (έξοδο). Η Sigmoid παράγει έξοδο στο διάστημα [0,1], προσφέροντας πιθανοτική ερμηνεία και είναι συμβατή με τη συνάρτηση απώλειας (Loss), η οποία απαιτεί οι έξοδοι να είναι πιθανότητες για binary classification.

Συγκεκριμένα, εφαρμόζω τη ReLU (Rectified Linear Unit) στα κρυφά επίπεδα και τη Sigmoid στο επίπεδο εξόδου.
Η επιλογή της ReLU στα ενδιάμεσα επίπεδα γίνεται επειδή επιταχύνει τη διαδικασία μάθησης, είναι υπολογιστικά αποδοτική και δεν εμφανίζει τόσο έντονα το πρόβλημα της εξασθένησης του παραγώγου (vanishing gradient). Επιπλέον, διατηρεί μόνο τις θετικές τιμές, κάτι που βοηθά στη δημιουργία αραιών και πιο αποδοτικών αναπαραστάσεων.

Από την άλλη, χρησιμοποιώ τη Sigmoid στο τελικό επίπεδο επειδή το πρόβλημα είναι δυαδικής ταξινόμησης και χρειάζομαι μια έξοδο που να εκφράζει πιθανότητα μεταξύ 0 και 1. Αυτό μου επιτρέπει να ερμηνεύσω εύκολα την πρόβλεψη του μοντέλου ως "πιθανότητα θετικής κλάσης".

Αρχικοποιούμε τα βάρη και το bias για κάθε επίπεδο του νευρωνικού δικτύου. Δηλαδή, για κάθε επίπεδο υπολογίζουμε πόσοι είναι οι νευρώνες εισόδου (rows) και πόσοι είναι οι νευρώνες εξόδου (cols) για το συγκεκριμένο επίπεδο. Με βάση αυτούς, υπολογίζουμε μια τιμή διασποράς std_dev που χρησιμοποιείται για να ελέγχει τις αρχικές τιμές των βαρών. (Είναι αναγκαίο γιατί σε περίπτωση που τα βάρη είναι πολύ μικρά, μπορεί να "σβήσουν" και το δίκτυο να μην μαθαίνει, και αν είναι πολύ μεγάλα, οι ενεργοποιήσεις μπορεί να "εκραγούν". Με το std_dev διασφαλίζουμε ότι τα βάρη θα αρχίζουν με σωστή τυπική απόκλιση, δηλαδή με μια τιμή που εξισορροπεί το μέγεθος των εισόδων και εξόδων κάθε επιπέδου). Δημιουργούμε έναν πίνακα βαρών rows*cols με τυχαίες τιμές από κανονική κατανομή και πολλαπλασιάζουμε κάθε στοιχείο του πίνακα με την τυπική απόκλιση, ώστε οι τελικές τιμές να έχουν κανονική κατανομή με τυπική απόκλιση std_dev. Τα bias b αρχικοποιούνται στο 0 σε έναν πίνακα με 1 γραμμή και τόσες στήλες όσοι είναι οι νευρώνες εξόδου (cols). Στο τέλος, τα βάρη και το bias αποθηκεύονται στις αντίστοιχες λίστες που δημιουργήσαμε για τα βάρη (self.weights) και για το bias (self.bias).

➢ Εκπαίδευση του Νευρωνικού Δικτύου

Πριν υλοποιήσουμε την εκπαίδευση του νευρωνικού δικτύου, δημιουργούμε τις μεθόδους που θα χρειαστούμε:

1. Forward Propagation (def forward)

Αυτή είναι η διαδικασία για τον **υπολογισμό της εξόδου του νευρωνικού δικτύου με βάση τις εισόδους**.

Αρχικά, η αρχική είσοδος (x) αποθηκεύεται σε μια λίστα (self.a), που αντιπροσωπεύει το επίπεδο εισόδου. Έπειτα, αυτή η λίστα αποθηκεύει τις εξόδους κάθε επιπέδου μετά την εφαρμογή της συνάρτησης ενεργοποίησης (για τις κρυφές στρώσεις χρησιμοποιούμε τη συνάρτηση ενεργοποίησης **ReLU**).
Παράλληλα, δημιουργείται μια άλλη λίστα (self.z), όπου αποθηκεύονται οι ενδιάμεσοι υπολογισμοί πριν την εφαρμογή της συνάρτησης ενεργοποίησης. Για κάθε κρυφό στρώμα γίνεται ο υπολογισμός των τιμών των νευρώνων του δικτύου (δηλαδή των εξόδων τους), έχοντας τις τιμές των εισόδων από το προηγούμενο στρώμα.

Μετά την επεξεργασία όλων των κρυφών επιπέδων, το επίπεδο εξόδου υπολογίζεται με τον ίδιο τρόπο όπως οι κρυφοί νευρώνες, αλλά με διαφορετική συνάρτηση ενεργοποίησης, τη **Sigmoid** (είναι ιδανική για προβλήματα δυαδικής ταξινόμησης). Τέλος, η μέθοδος forward επιστρέφει την τελική ενεργοποίηση του επιπέδου εξόδου (self.a[-1]), η οποία είναι η προβλεπόμενη πιθανότητα από το δίκτυο για το δοσμένο δείγμα εισόδου.

2. Back Propagation (def backward)

Αφού το δίκτυο δώσει μια πρόβλεψη (μέσω της forward μεθόδου) και αυτή συγκριθεί με τη σωστή απάντηση, η backward ξεκινά μια διαδικασία "οπισθοδρόμησης" για να βρει πού έγιναν λάθη και πώς να τα διορθώσει.

Μετά την πρόβλεψη του δικτύου (y_pred) για την είσοδο (x), το πρώτο κρίσιμο βήμα είναι ο υπολογισμός του **delta (σφάλματος)** για το επίπεδο εξόδου. Αυτό γίνεται αφαιρώντας τη σωστή τιμή (y_true) από την πρόβλεψη του δικτύου (y_pred) και το σφάλμα αυτό πολλαπλασιάζεται με την παράγωγο της συνάρτησης **Sigmoid** (της συνάρτησης ενεργοποίησης του επιπέδου εξόδου).
Στη συνέχεια, το σφάλμα αυτό "διαδίδεται" προς τα πίσω, από το επίπεδο εξόδου σε κάθε προηγούμενο κρυφό επίπεδο. Σε κάθε επίπεδο, το δίκτυο υπολογίζει πόσο συνέβαλε αυτό το συγκεκριμένο επίπεδο στο συνολικό σφάλμα. Καθώς το σφάλμα διαδίδεται προς τα πίσω, το δίκτυο υπολογίζει τις **"οδηγίες διόρθωσης" (gradients)** για κάθε βάρος και bias. Αυτές οι οδηγίες δείχνουν πόσο και προς ποια κατεύθυνση πρέπει να αλλάξει κάθε βάρος και bias.

3. Update (def update)

Αφού η μέθοδος backward έχει υπολογίσει με ακρίβεια πόσο "λάθος" ήταν η πρόβλεψη του δικτύου, η μέθοδος update περνάει συστηματικά από όλες τις συνδέσεις (δηλαδή, από όλα τα ζεύγη βαρών και biases) σε όλα τα επίπεδα του νευρωνικού δικτύου. Για κάθε σύνολο βαρών σε κάθε επίπεδο, υπολογίζει τα καινούργια βάρη και bias.

Η εκπαίδευση του νευρωνικού δικτύου συντονίζει όλη τη διαδικασία μάθησης, χρησιμοποιώντας τις μεθόδους **εμπρόσθια διάδοση (forward propagation)**, **αντίστροφη διάδοση (back propagation)** και **ενημέρωση βαρών και bias**.

Η μέθοδος train λαμβάνει ως αρχικές εισόδους τα δεδομένα εκπαίδευσης (X) και τις σωστές απαντήσεις (y) για αυτά τα δεδομένα, τον συνολικό αριθμό των **εποχών (epochs)** που θα διαρκέσει η εκπαίδευση, τον **ρυθμό μάθησης (lr)** και μια παράμετρο **verbose** για την εμφάνιση της προόδου, αν εντοπίστηκαν προβλήματα και για να αποκτήσουμε μια εικόνα για αυτά.
*Έχω ορίσει **100 εποχές**, διότι μετά από κάποιες δοκιμές (π.χ., 90, 200 και 1000 εποχές) εντοπίζω πως η ακρίβεια κατά τη διάρκεια των εποχών σταθεροποιείται κάπου μεταξύ 85-90 εποχών. Επιπλέον, στις 100 εποχές και με ρυθμό μάθησης (lr=0.03), ο νευρώνας έχει την καλύτερη ακρίβεια **0.9850** σε σύγκριση με άλλες προσπάθειες και εκτελείται πιο γρήγορα.*

Μέσα σε κάθε εποχή, η μέθοδος train εξετάζει τα δεδομένα ένα δείγμα κάθε φορά (γνωστό ως **stochastic gradient descent**).

Για κάθε δείγμα που δίνουμε στο νευρωνικό δίκτυο, το πρώτο πράγμα που κάνει είναι να το "περάσει" μέσα από τα εσωτερικά του στρώματα για να καταλάβει τι προβλέπει. Αυτό γίνεται με τη μέθοδο **self.backward**, η οποία στην αρχή καλεί τη **self.forward**. Η forward είναι αυτή που πραγματικά κάνει την πρόβλεψη, περνώντας το δείγμα από όλα τα στρώματα και βγάζοντας ένα αποτέλεσμα (0 ή 1).
Αφού το δίκτυο κάνει την πρόβλεψή του, η μέθοδος **backward** βλέπει αν το αποτέλεσμα ήταν σωστό και πώς να διορθώσει αυτό το λάθος. Δηλαδή, υπολογίζει πόσο και προς ποια κατεύθυνση πρέπει να αλλάξει κάθε βάρος και κάθε bias μέσα στο δίκτυο, για να γίνει καλύτερο την επόμενη φορά.
Εφόσον έχουν υπολογιστεί οι οδηγίες διόρθωσης, η μέθοδος **update** προσαρμόζει τα βάρη και το bias ακολουθώντας τις κλίσεις που υπολογίστηκαν. Το πόσο μεγάλη θα είναι αυτή η προσαρμογή το καθορίζει ο **ρυθμός μάθησης(lr)**.
Αν ο ρυθμός μάθησης είναι μικρός, οι αλλαγές γίνονται προσεκτικά και αργά, ώστε να μη "ξεφύγει" το δίκτυο. Αν είναι μεγάλος, οι αλλαγές γίνονται πιο γρήγορα, αλλά υπάρχει και μεγαλύτερος κίνδυνος να γίνουν λάθος βήματα και να μην μπορέσει το δίκτυο να μάθει σωστά.

Μόλις ολοκληρωθεί η επεξεργασία όλων των δειγμάτων για την τρέχουσα εποχή, υπολογίζεται η **μέση απώλεια** και η **συνολική ακρίβεια** για αυτή την εποχή. Αυτές οι τιμές αποθηκεύονται σε δύο λίστες, ώστε να μπορούμε να παρακολουθούμε πώς τα πάει το δίκτυο με το πέρασμα του χρόνου. Εάν η παράμετρος **verbose** είναι ενεργοποιημένη, η μέθοδος εκτυπώνει κάθε λίγες εποχές πόσο καλά τα πάει το δίκτυο εκείνη τη στιγμή.
Όταν τελειώσουν όλες οι εποχές εκπαίδευσης του δικτύου , η μέθοδος **train** επιστρέφει μια λίστα με το **λάθος (loss)** σε κάθε εποχή και μια λίστα με την **ακρίβεια (accuracy)** σε κάθε εποχή.

➢ Αξιολόγηση στο Σύνολο Δοκιμών

Ένα σημαντικό κομμάτι για την υλοποίηση του νευρωνικού δικτύου είναι η **τελική αξιολόγησή του στο σύνολο δοκιμών (test set evaluation)**. Αφού το δίκτυο έχει "μάθει" από τα δεδομένα εκπαίδευσης (xTrain, yTrain), πρέπει να ελέγξουμε αν μπορεί να εφαρμόσει αυτές τις γνώσεις σε νέα δεδομένα που δεν έχει δει (test set evaluation).
Αυτό το καταφέρνουμε συλλέγοντας τις **δυαδικές προβλέψεις (0 ή 1)** από το εκπαιδευμένο νευρωνικό δίκτυο για κάθε δείγμα του συνόλου δοκιμών **(xTest)**. Το μοντέλο παίρνει κάθε δείγμα ξεχωριστά, κάνει την πρόβλεψή του (αν ανήκει στην κατηγορία 0 ή 1) και αποθηκεύει αυτές τις προβλέψεις σε μια λίστα.
Στη συνέχεια, υπολογίζουμε την **ακρίβεια (accuracy)** του μοντέλου στο σύνολο δοκιμών. Αυτό επιτυγχάνεται συγκρίνοντας στοιχείο προς στοιχείο τις προβλέψεις **(y_pred_test)** με τις πραγματικές ετικέτες **(yTest)**. Το αποτέλεσμα είναι ένας πίνακας από **True** και **False** τιμές (όπου True σημαίνει σωστή πρόβλεψη).
Υπολογίζουμε τον μέσο όρο αυτού του πίνακα, ο οποίος αντιπροσωπεύει απευθείας το ποσοστό των σωστών προβλέψεων, δηλαδή την ακρίβεια.

➢ Οπτικοποίηση των Αποτελεσμάτων

1. **Οπτικοποίηση Απώλειας και Ακρίβειας κατά την Εκπαίδευση**

Δημιουργούμε δύο ξεχωριστά γραφήματα σε μία κοινή εικόνα. Το πρώτο γράφημα δείχνει την εξέλιξη της **απώλειας (loss)** του δικτύου σε κάθε εποχή εκπαίδευσης. Μια επιτυχημένη εκπαίδευση συνήθως απεικονίζεται από μια καμπύλη απώλειας που **μειώνεται σταθερά** με την πάροδο των εποχών. Αυτό υποδηλώνει ότι το δίκτυο προσαρμόζεται σωστά στα δεδομένα και κάνει όλο και μικρότερα λάθη.
Παράλληλα, το δεύτερο γράφημα απεικονίζει την **ακρίβεια (accuracy)** του δικτύου. Μια **αυξητική πορεία** της καμπύλης ακρίβειας είναι το επιθυμητό αποτέλεσμα, καθώς δείχνει ότι το δίκτυο κάνει όλο και περισσότερες σωστές προβλέψεις.

2. **Οπτικοποίηση της Περιοχής Απόφασης (Decision Boundary)**

Για να δούμε πώς "σκέφτεται" το μοντέλο για το ποια σημεία ανήκουν σε κάθε κατηγορία, αρχικά δημιουργούμε ένα **πλέγμα σημείων** που καλύπτει όλο τον χώρο γύρω από τα δεδομένα. Δηλαδή, φτιάχνουμε πολλά τεχνητά σημεία σε όλο το επίπεδο. Για κάθε σημείο, το μοντέλο κάνει μια πρόβλεψη. Μετά, το γράφημα χρωματίζεται σύμφωνα με αυτές τις προβλέψεις. Έτσι, προκύπτει ένα έγχρωμο φόντο. Πάνω σε αυτό το φόντο, σχεδιάζονται και τα πραγματικά σημεία του test set, ώστε να δούμε αν το μοντέλο χώρισε σωστά τις δύο ομάδες.

3. Αναλυτική Αναφορά Ταξινόμησης

Η **αναφορά ταξινόμησης (classification report)** είναι σαν ένα λεπτομερές δελτίο ελέγχου για το πόσο καλά αποδίδει το μοντέλο.

<u>Αρχικά, υπολογίζουμε πόσες προβλέψεις ήταν σωστές ή λάθος, χωρίζοντάς τες σε:</u>

True Positives (TP): Το μοντέλο είπε "ναι" και ήταν όντως "ναι".
True Negatives (TN): Το μοντέλο είπε "όχι" και ήταν όντως "όχι".
False Positives (FP): Το μοντέλο είπε "ναι", αλλά ήταν λάθος.
False Negatives (FN): Το μοντέλο είπε "όχι", αλλά έπρεπε να πει "ναι".

<u>Με βάση αυτά τα αποτελέσματα, υπολογίζουμε τα βασικά στατιστικά:</u>

Ακρίβεια (Accuracy): Αυτό είναι το συνολικό ποσοστό των σωστών προβλέψεων. Είναι η "γενική εικόνα" της επιτυχίας του μοντέλου.

Precision (Ακρίβεια Θετικών Προβλέψεων): Το Precision μάς λέει πόσες φορές το μοντέλο είχε δίκιο όταν είπε "ναι". Δηλαδή, όταν το μοντέλο προβλέπει κάτι ως θετικό, πόσες φορές αυτό ήταν όντως σωστό. Είναι σημαντικό γιατί μας δείχνει πόσο αξιόπιστο είναι το μοντέλο όταν προβλέπει κάτι ως "θετικό".

Recall (Ανάκληση / Ευαισθησία): Το Recall μάς λέει πόσες από τις πραγματικά θετικές περιπτώσεις κατάφερε το μοντέλο να βρει. Δηλαδή, όταν έπρεπε να πει "ναι", πόσες φορές το είπε σωστά.

Specificity (Ειδικότητα): Αυτή η μέτρηση είναι το "Recall" για την αρνητική κλάση. Μας λέει "Από όλες τις φορές που κάτι ήταν πραγματικά 'αρνητικό', πόσες από αυτές τις περιπτώσεις τις αναγνωρίσαμε σωστά ως 'αρνητικές'".
F1 Score (F1 Βαθμολογία): Είναι ένας **συνδυασμός** του Precision και του Recall. Δίνει μια ισορροπημένη εικόνα της απόδοσης, ειδικά όταν το Precision και το Recall είναι ανισόρροπα μεταξύ τους.

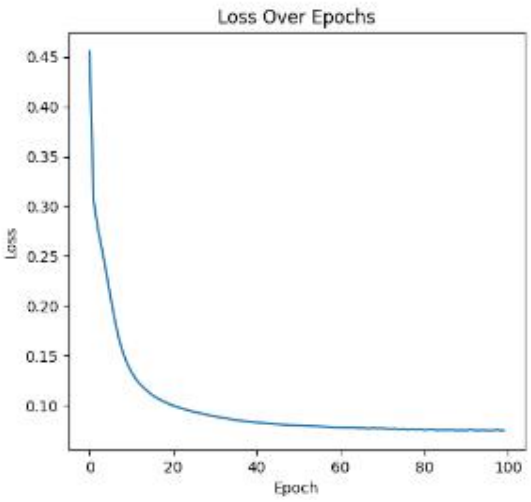
Πληροφορίες (Support): Το τμήμα "Support" σας ενημερώνει για το πλήθος των δειγμάτων που ανήκουν πραγματικά σε κάθε κατηγορία στο σύνολο δεδομένων που αξιολογήθηκε.

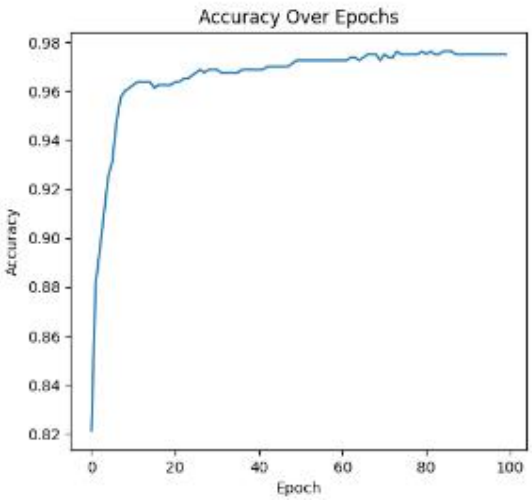
Positive class: Πόσα δείγματα ήταν πραγματικά θετικά.
Negative class: Πόσα δείγματα ήταν πραγματικά αρνητικά.
Total samples: Το σύνολο των δειγμάτων που χρησιμοποιήθηκαν για την αξιολόγηση.

Συνοπτικα :

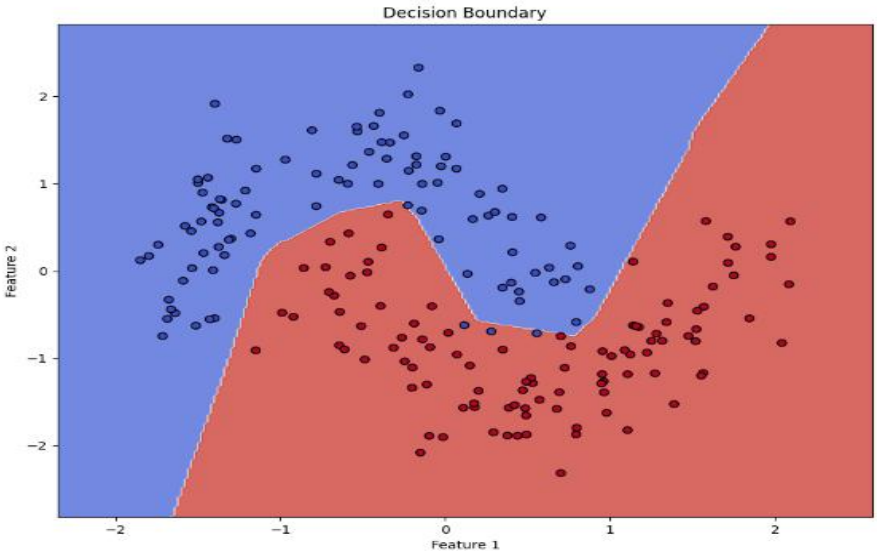
Στην παρούσα υλοποίηση, το MLP αποτελείται από μία είσοδο δύο διαστάσεων, δύο κρυφές στρώσεις με 10 νευρώνες η κάθε μία και μία έξοδο. Οι κρυφές στρώσεις χρησιμοποιούν τη συνάρτηση ενεργοποίησης **ReLU**, ενώ η έξοδος χρησιμοποιεί τη **Sigmoid.**Το δίκτυο εκπαιδεύεται με **Stochastic Gradient Descent (SGD)** και συνάρτηση κόστους τη **Binary Cross-Entropy**. Χρησιμοποιείται **ρυθμός μάθησης 0.03** και **100 εποχές.** Και για το MLP αυτό έχουμε :

- Πίνακες accuracy, loss





- Διαγράμματα σύγκρισης (matplotlib)



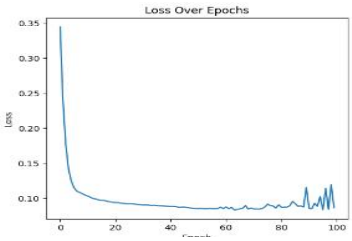
- Αναφορά Ταξινόμησης

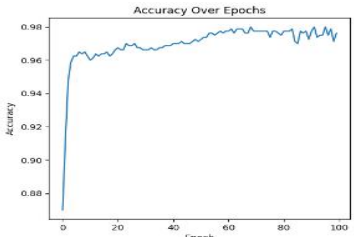
```
Test Accuracy: 0.9850

 Enhanced Classification Report
----------------------------------------
Accuracy    : 0.9850
Precision   : 0.9720
Recall      : 1.0000
Specificity : 0.9687
F1 Score    : 0.9858

 Support:
  Positive class: 104
  Negative class: 96
  Total samples: 200
```

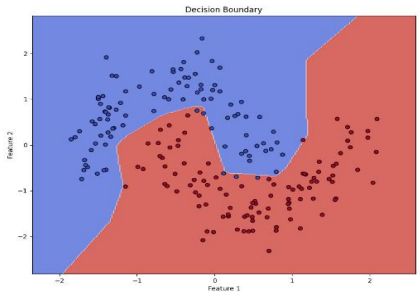
➢ Πειραματα που εκτελεστηκαν :

❖ Εκπαίδευση με διαφορετικά learning rates

Δοκημη 1 : Learning rate **(lr) = 0.1**

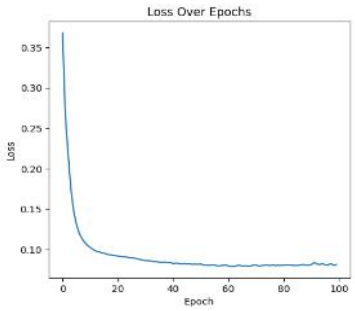


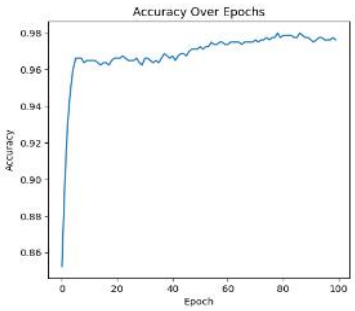


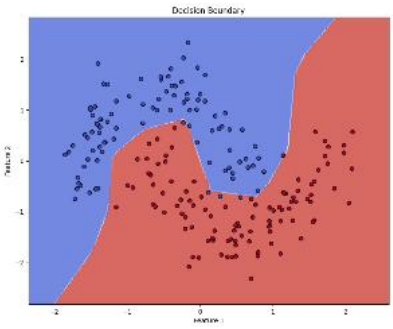



```
Enhanced Classification Report
--------------------------------------
Accuracy    : 0.9700
Precision   : 0.9537
Recall      : 0.9904
Specificity : 0.9479
F1 Score    : 0.9717

Support:
 Positive class: 104
 Negative class: 96
 Total samples: 200
```

Δοκημη 2 : Learning rate **(lr) = 0.07**

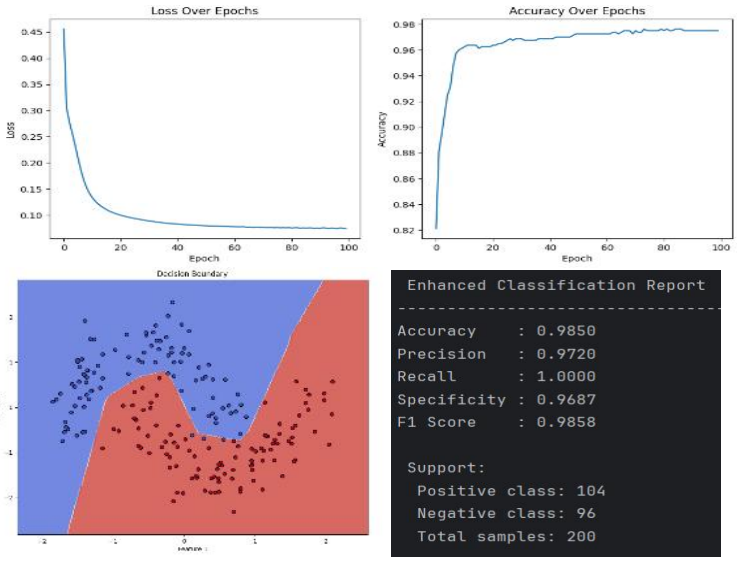




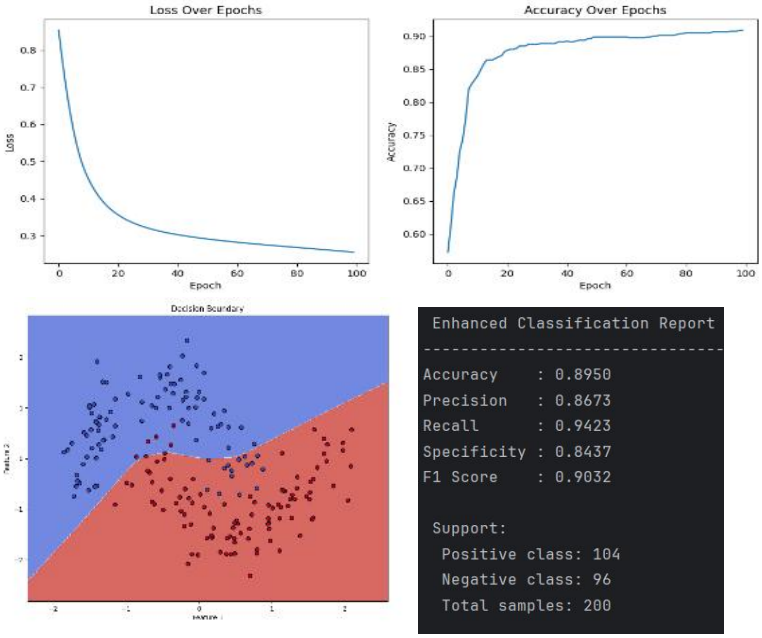



```
Enhanced Classification Report
--------------------------------
Accuracy    : 0.9750
Precision   : 0.9626
Recall      : 0.9904
Specificity : 0.9583
F1 Score    : 0.9763

Support:
 Positive class: 104
 Negative class: 96
 Total samples: 200
```

Δοκημη 3 : Learning rate **(lr) = 0.01**



Δοκημη 4 : Learning rate (lr) = 0.001



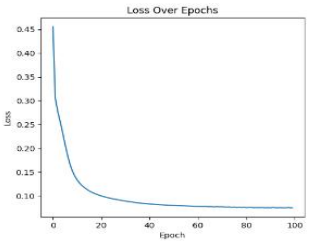
Από τα πειράματα, διαπίστωσα πως ο ιδανικός ρυθμός μάθησης **(learning rate)** είναι το **0.01** ή το **0.03(** όπως χρησιμοποιώ). Όταν ο ρυθμός μάθησης είναι πολύ μεγάλος (**0.07**), το MLP δεν μαθαίνει καλά και παρουσιάζει αστάθεια. Αυτό φάνηκε στα αποτελέσματα, όπου το **loss (απώλεια)** ανεβοκατέβαινε έντονα, το **Accuracy (ακρίβεια)** δεν αυξανόταν σταθερά, και η απόδοση στο test set χειροτέρευε. Το ίδιο συμβαίνει και με lr=0.07.

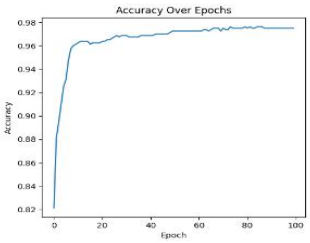
Όταν, όμως, το lr είναι πολύ μικρό (π.χ., 0.001), το MLP φαίνεται να μην μαθαίνει τίποτα. Αυτό συμβαίνει διότι, με τόσο μικρό ρυθμό μάθησης, το μοντέλο κάνει ελάχιστη πρόοδο σε κάθε εποχή και για να λειτουργήσει σωστά ο αλγόριθμος

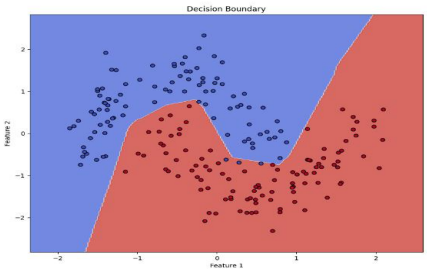
χρειάζεται πάρα πολλές εποχές (π.χ., 2000+ εποχές). Η ανάγκη για **πάρα πολλές εποχές** (π.χ., 2000+) καθιστά τον αλγόριθμο **εξαιρετικά αργό** και μη πρακτικό.Αυτό είναι κάτι που σίγουρα δε θέλουμε στην πράξη, καθώς ο χρόνος εκπαίδευσης είναι ένας κρίσιμος παράγοντας στην ανάπτυξη μοντέλων μηχανικής μάθησης.

❖ Σύγκριση συναρτήσεων ενεργοποίησης:

1-Με **ReLu** και **Sigmoid**

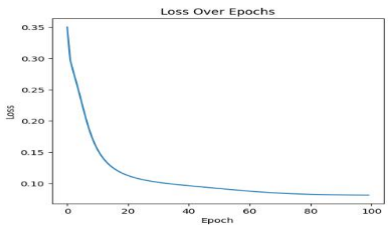


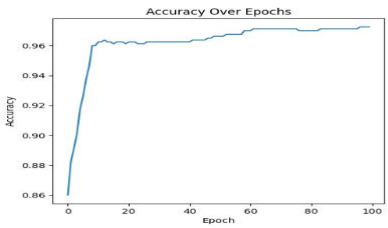


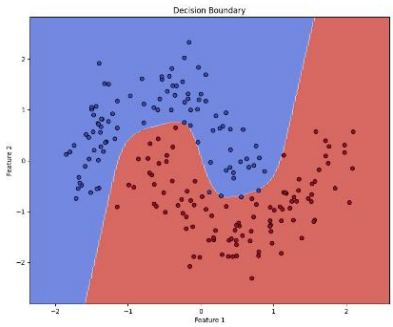



```
Enhanced Classification Report
--------------------------------------
Accuracy    : 0.9850
Precision   : 0.9720
Recall      : 1.0000
Specificity : 0.9687
F1 Score    : 0.9858

Support:
  Positive class: 104
  Negative class: 96
  Total samples: 200
```

2- Με **tanh** και **Sigmoid :**



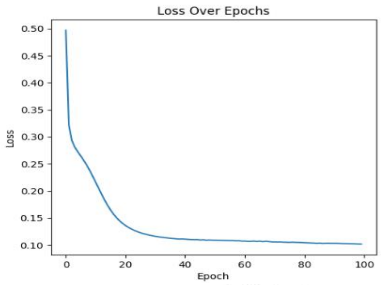


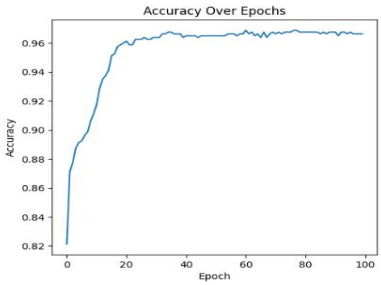


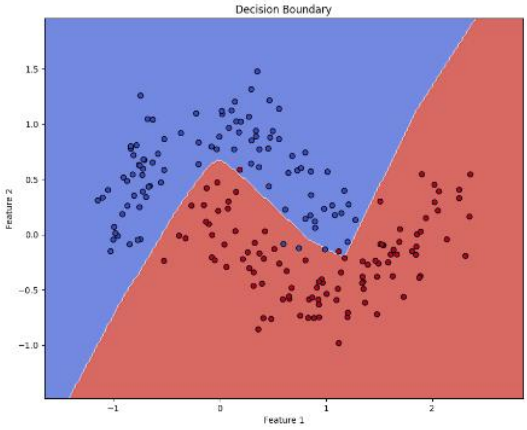

```
Enhanced Classification Report
--------------------------------------
Accuracy    : 0.9850
Precision   : 0.9720
Recall      : 1.0000
Specificity : 0.9687
F1 Score    : 0.9858

Support:
  Positive class: 104
  Negative class: 96
  Total samples: 200
```

Όταν συγκρίνουμε τις συναρτήσεις ενεργοποίησης, τόσο η **Tanh** όσο και η **ReLU** (σε συνδυασμό με τη Sigmoid στην έξοδο), οδηγούν σε παρόμοια τελικά αποτελέσματα. Ωστόσο, η **ReLU** συχνά ξεχωρίζει γιατί η ακρίβεια του μοντέλου αυξάνεται με έναν πιο **ομαλό και σταθερό ρυθμό** σε κάθε εποχή εκπαίδευσης.

❖ <u>Σύγκριση αποτελεσμάτων με και χωρίς κανονικοποίηση:</u>








```
Test Accuracy: 0.9750

 Enhanced Classification Report
---------------------------------------
Accuracy    : 0.9750
Precision   : 0.9714
Recall      : 0.9808
Specificity : 0.9687
F1 Score    : 0.9761

 Support:
  Positive class: 104
  Negative class: 96
  Total samples: 200
```

Παρόλο που η κανονικοποίηση βοηθάει σημαντικά, σε ορισμένες περιπτώσεις μπορεί το MLP να λειτουργήσει ικανοποιητικά και χωρίς αυτήν.
Το make_moons είναι ένα **απλό και σχετικά ισορροπημένο** dataset (με 2 διαστάσεις και χωρίς τεράστιες διαφορές κλίμακας).Τα χαρακτηριστικά του δεν έχουν πολύ μεγάλη διασπορά, οπότε ακόμα και χωρίς κανονικοποίηση, οι ενεργοποιήσεις δεν εκρήγνυνται ή μηδενίζονται.

Συμπεράσματα – Τεκμηρίωση Ορθότητας:

Η επιλογή ενός πολυεπίπεδου νευρωνικού δικτύου με δύο κρυφές στρώσεις των 10 νευρώνων είναι κατάλληλη για την επίλυση προβλημάτων δυαδικής ταξινόμησης σε **μη-γραμμικά διαχωρίσιμα δεδομένα**, όπως το dataset "make moons". Η πολυπλοκότητα αυτής της αρχιτεκτονικής επαρκεί για να μοντελοποιήσει τις σύνθετες σχέσεις στα δεδομένα χωρίς να οδηγήσει σε υπερβολική προσαρμογή.

Η χρήση της **ReLU** στα κρυφά επίπεδα και της **Sigmoid** στο επίπεδο εξόδου είναι μια βέλτιστη πρακτική. Η ReLU επιτρέπει την ομαλή ροή των κλίσεων (gradients) κατά την οπισθοδιάδοση, αντιμετωπίζοντας το πρόβλημα του "vanishing gradient" και εξασφαλίζοντας σταθερή αύξηση της ακρίβειας. Η Sigmoid, από την άλλη, είναι ιδανική για δυαδική ταξινόμηση, καθώς μετατρέπει την έξοδο σε πιθανότητες στο διάστημα [0,1], συμβατή με τη συνάρτηση απώλειας δυαδικής εντροπίας.

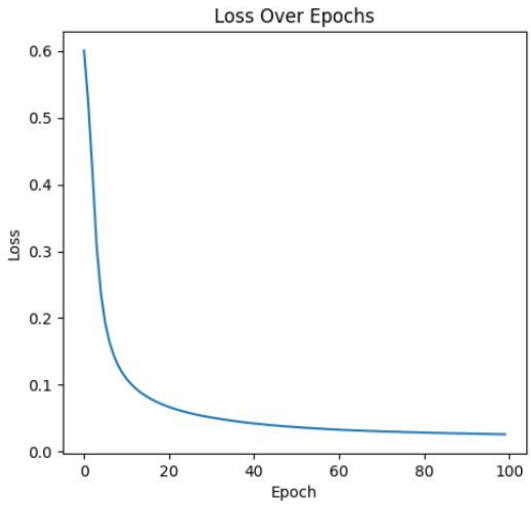
Οι 100 εποχές αποδείχθηκαν επαρκείς. Όπως διαπιστώθηκε, η ακρίβεια σταθεροποιείται γύρω στις 85-90 εποχές, υποδεικνύοντας ότι περαιτέρω εκπαίδευση δεν προσφέρει σημαντική βελτίωση και αυξάνει άσκοπα τον υπολογιστικό χρόνο. Η πειραματική εύρεση του lr στο εύρος 0.01 - 0.03 είναι κρίσιμη. Τιμές εκτός αυτού του εύρους οδήγησαν σε προβλήματα

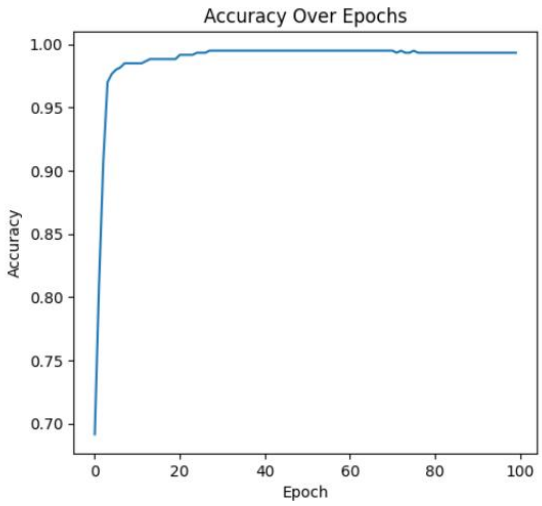
Συνολικά, ο κώδικας είναι δομημένος και υλοποιημένος με τρόπο που ακολουθεί τις καθιερωμένες αρχές των νευρωνικών δικτύων, επιδεικνύοντας **υψηλή ακρίβεια (0.9850)** στο σύνολο δοκιμών, γεγονός που τεκμηριώνει την ορθότητά του και την αποτελεσματική επίλυση του προβλήματος δυαδικής ταξινόμησης.

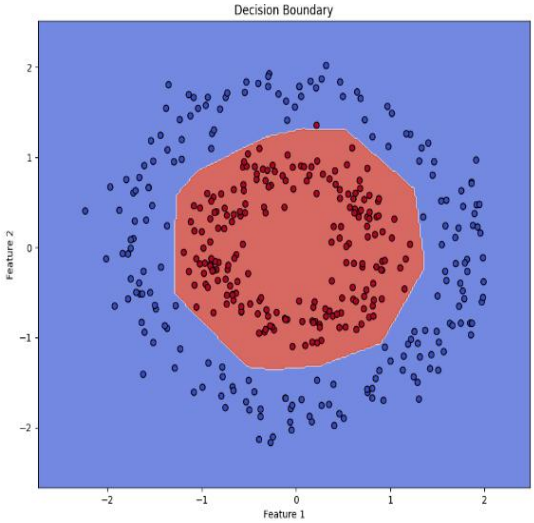
Bonus 1 : <u>Αξιοποιείστε διαφορετικό data set για να υλοποιείστε το MLP δίκτυο σας.</u>

Στο πλαίσιο των δοκιμών, πραγματοποίησα επίσης την υλοποίηση του νευρωνικού δικτύου με το dataset make_circles από το sklearn.datasets. Η αρχιτεκτονική του MLP παρέμεινε η ίδια, με τη μοναδική διαφοροποίηση ότι για το συγκεκριμένο dataset όρισα το **60% των δεδομένων για την εκπαίδευση** και το **40% για τη δοκιμή**.

Τα αποτελέσματα έδειξαν πως το νευρωνικό δίκτυο λειτούργησε εξίσου αποτελεσματικά.








```
Test Accuracy: 0.9975

 Enhanced Classification Report
---------------------------------------
Accuracy    : 0.9975
Precision   : 1.0000
Recall      : 0.9951
Specificity : 1.0000
F1 Score    : 0.9976

 Support:
  Positive class: 205
  Negative class: 195
  Total samples: 400
```